RoomBooker Kiosk: Comprehensive System Specification

This document serves as the master architectural specification, implementation blueprint, and deployment playbook for the **RoomBooker Kiosk** system.

1. System Overview & Objectives

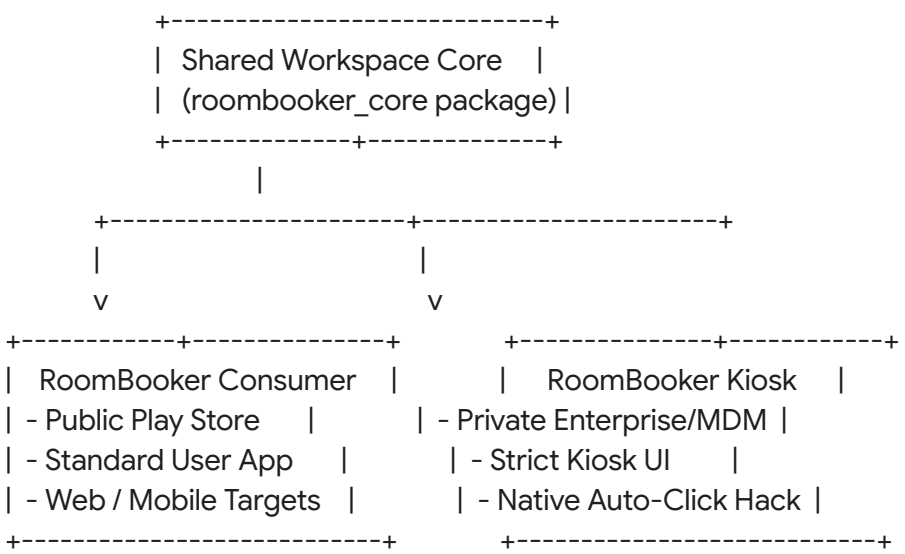
The **RoomBooker Kiosk** is a hardware-bound, tablet-based scheduling and video conferencing terminal designed for 24/7 wall-mounted operation outside meeting rooms.

Key Capabilities

- **One-Touch Join:** Launches native clients for Google Meet, Microsoft Teams, and Zoom, automatically bypassing pre-meeting "green room" or "lobby" preview screens.
- **Hardware-Class Reliability (Path A):** Utilizes battery-less, commercial-grade PoE hardware (Mimo Monitors) to completely eliminate battery safety risks (bloat) and simplify cabling.
- **Kiosk Lockdown:** Restricts user interaction purely to the RoomBooker dashboard via Android's native LockTaskMode.
- **Physical Status Indicators:** Leverages programmable LED side lights on the physical tablet to visually project room availability (Green/Red) directly down the hallway.

Two-App Architecture Model

To preserve a clean public profile for the client-facing app and bypass Google's strict public Play Store regulations regarding accessibility automation, the project is divided into two separate applications:



2. Directory & Workspace Structure

The project is structured as a Flutter monorepo workspace. The common data models, authentication handlers, and Firestore repository services live in roombooker_core, while presentation logic is split.

```
roombooker-workspace/
├── packages/
│   ├── roombooker_core/       # Shared models, repository layers, Firestore adapters
│   ├── roombooker_consumer/    # Standard public mobile/web client
│   └── roombooker_kiosk/      # Dedicated hardware kiosk app
│       ├── android/
│       │   └── app/
│       │       └── src/
│       │           └── main/
│       │               ├── kotlin/com/example/roombooker_kiosk/
│       │               │   ├── MainActivity.kt
│       │               │   └── MeetAutomatorService.kt
│       │               └── res/
│       │                   └── xml/
│       │                       └── accessibility_service_config.xml
│       └── lib/
│           ├── main.dart      # App entrypoint initializing LockTaskMode
│           └── views/
│               └── kiosk_dashboard.dart  # Tablet dashboard with real-time room feeds
```

3. Native Android Configuration Layer

To authorize the background automation engine, specific permissions and services must be registered inside the Android sub-project of roombooker_kiosk.

3.1 Accessibility Configuration Rules

File: android/app/src/main/res/xml/accessibility_service_config.xml

```
<?xml version="1.0" encoding="utf-8"?>
<accessibility-service xmlns:android="http://schemas.android.com/apk/res/android"
    android:description="@string/accessibility_description"
    android:accessibilityEventTypes="typeWindowStateChanged|typeWindowContentChanged"
    android:packageNames="com.google.android.apps.meetings,com.microsoft.teams"
    android:accessibilityFeedbackType="feedbackGeneric"
```

```
    android:notificationTimeout="100"
    android:canRetrieveWindowContent="true"
    android:canPerformGestures="false" />
```

3.2 Resource Strings

File: android/app/src/main/res/values/strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">RoomBooker Kiosk</string>
    <string name="accessibility_description">Automates conference room check-in and
confirmation actions for managed hardware displays.</string>
</resources>
```

3.3 Manifest Configurations

File: android/app/src/main/AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- Explicit package queries for Android 11+ security constraints -->
    <queries>
        <package android:name="com.google.android.apps.meetings" />
        <package android:name="com.microsoft.teams" />
        <package android:name="us.zoom.videomeetings" />
    </queries>

    <application
        android:label="RoomBooker Kiosk"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:launchMode="singleTask"
            android:theme="@style/LaunchTheme"

android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smallestScreenSize|
screenLayout|density|uiMode"
            android:hardwareAccelerated="true"
```

```
            android:windowSoftInputMode="adjustResize">
            <meta-data
              android:name="io.flutter.embedding.android.NormalTheme"
              android:resource="@style/NormalTheme" />
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>

        <!-- Native UI Click Automation Service -->
        <service android:name=".MeetAutomatorService"
            android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE"
            android:exported="true">
            <intent-filter>
                <action android:name="android.view.accessibility.AccessibilityService" />
            </intent-filter>
            <meta-data
                android:name="android.view.accessibility.AccessibilityService"
                android:resource="@xml/accessibility_service_config" />
        </service>

    </application>
</manifest>
```

4. Backend Native Automation Engine

The MeetAutomatorService runs as a persistent background helper. When Google Meet or Microsoft Teams enters the foreground, it inspects the window tree, finds the target click targets, and programmatically executes a physical click on the layout bounds.

File: android/app/src/main/kotlin/com/example/roombooker_kiosk/MeetAutomatorService.kt

```
package com.example.roombooker_kiosk

import android.accessibilityservice.AccessibilityService
import android.view.accessibility.AccessibilityEvent
import android.view.accessibility.AccessibilityNodeInfo
import android.util.Log

class MeetAutomatorService : AccessibilityService() {

    private val TAG = "MeetAutomator"
```

```
override fun onAccessibilityEvent(event: AccessibilityEvent) {
    val packageName = event.packageName?.toString() ?: return
    val rootNode = rootInActiveWindow ?: return

    // Targeted action-oriented labels per application type
    val targetedButtons = when (packageName) {
        "com.google.android.apps.meetings" -> listOf("Join", "Join meeting", "Join now")
        "com.microsoft.teams" -> listOf("Join now", "Join")
        else -> return
    }

    findAndExecuteClick(rootNode, targetedButtons)
}

private fun findAndExecuteClick(node: AccessibilityNodeInfo?, targets: List<String>) {
    if (node == null) return

    val nodeText = node.text?.toString()?.trim()
    val nodeDesc = node.contentDescription?.toString()?.trim()

    for (target in targets) {
        if (nodeText?.equals(target, ignoreCase = true) == true ||
            nodeDesc?.equals(target, ignoreCase = true) == true) {

            var clickableNode: AccessibilityNodeInfo? = node
            // Traverse layout tree parents until reaching a clickable container element
            while (clickableNode != null && !clickableNode.isClickable) {
                clickableNode = clickableNode.parent
            }

            if (clickableNode != null && clickableNode.isClickable) {
                val success = clickableNode.performAction(AccessibilityNodeInfo.ACTION_CLICK)
                if (success) {
                    Log.i(TAG, "Successfully bypassed landing page for target text: $target")
                }
                return
            }
        }
    }

    // Search recursively across children
    for (i in 0 until node.childCount) {
```

```
            val child = node.getChild(i)
            findAndExecuteClick(child, targets)
        }
    }

    override fun onInterrupt() {
        Log.w(TAG, "Accessibility service interrupted.")
    }
}
```

5. Flutter Kiosk Implementation

5.1 Main Entrypoint

File: packages/roombooker_kiosk/lib/main.dart

```
import 'package:flutter/material';
import 'package:kiosk_mode/kiosk_mode.dart';
import 'views/kiosk_dashboard.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Enforce system Lock Task Mode to lock down OS inputs
  try {
    startKioskMode();
  } catch (error) {
    debugPrint("Failed to initialize hardware lockdown: $error");
  }

  runApp(const RoomBookerKioskApp());
}

class RoomBookerKioskApp extends StatelessWidget {
  const RoomBookerKioskApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'RoomBooker Kiosk Display',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
```

```
      brightness: Brightness.dark,
      primarySwatch: Colors.blueGrey,
      useMaterial3: true,
    ),
    home: const KioskDashboardView(),
   );
  }
}
```

5.2 Kiosk Dashboard & Intent Router

File: packages/roombooker_kiosk/lib/views/kiosk_dashboard.dart

```
import 'package:flutter/material';
import 'package:android_intent_plus/android_intent.dart';

class KioskDashboardView extends StatelessWidget {
 const KioskDashboardView({super.key});

 void _executePlatformIntent(BuildContext context, String rawUrl) async {
  String? nativePackage;
  String processedDataUrl = rawUrl;

  if (rawUrl.contains('meet.google.com')) {
   nativePackage = 'com.google.android.apps.meetings';
  } else if (rawUrl.contains('teams.microsoft.com') || rawUrl.contains('teams.live.com')) {
   nativePackage = 'com.microsoft.teams';
   // Format to force teams app to catch protocol schemes bypassing browser
   processedDataUrl = rawUrl.replaceAll('https://', 'msteams://');
  } else if (rawUrl.contains('zoom.us')) {
   nativePackage = 'us.zoom.videomeetings';
  }

  final AndroidIntent intent = AndroidIntent(
   action: 'action_view',
   data: processedDataUrl,
   package: nativePackage,
  );

  try {
   await intent.launch();
  } catch (exception) {
   if (context.mounted) {
```

```
      ScaffoldMessenger.of(context).showSnackBar(
       SnackBar(content: Text('Target client app not found: $exception')),
      );
     }
    }
   }

  @override
  Widget build(BuildContext context) {
   // Replace with local firestore model parameters in production
   const String sampleGoogleMeetUrl = "https://meet.google.com/abc-defg-hij";

   return Scaffold(
    body: Row(
     children: [
      // Left Info column
      Expanded(
       flex: 4,
       child: Container(
        color: const Color(0xFF151515),
        padding: const EdgeInsets.all(32),
        child: const Column(
         crossAxisAlignment: CrossAxisAlignment.start,
         mainAxisAlignment: MainAxisAlignment.center,
         children: [
          Text("CONFERENCE ROOM A", style: TextStyle(fontSize: 28, fontWeight:
FontWeight.bold)),
          SizedBox(height: 16),
          Text("Status: Available", style: TextStyle(fontSize: 18, color: Colors.greenAccent)),
         ],
        ),
       ),
      ),
      // Right CTA column
      Expanded(
       flex: 6,
       child: Padding(
        padding: const EdgeInsets.all(48.0),
        child: Center(
         child: SizedBox(
          width: double.infinity,
          height: 120,
          child: ElevatedButton.icon(
```

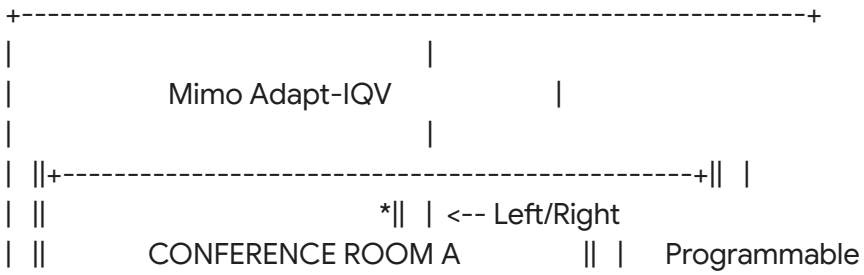
```
                style: ElevatedButton.styleFrom(
                  backgroundColor: Colors.blueAccent,
                  foregroundColor: Colors.white,
                  shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
                ),
                icon: const Icon(Icons.video_call_rounded, size: 48),
                label: const Text("ONE-TOUCH JOIN MEETING", style: TextStyle(fontSize: 22,
fontWeight: FontWeight.bold)),
                onPressed: () => _executePlatformIntent(context, sampleGoogleMeetUrl),
              ),
            ),
          ),
        ),
      ),
    ],
  ),
 );
 }
}
```

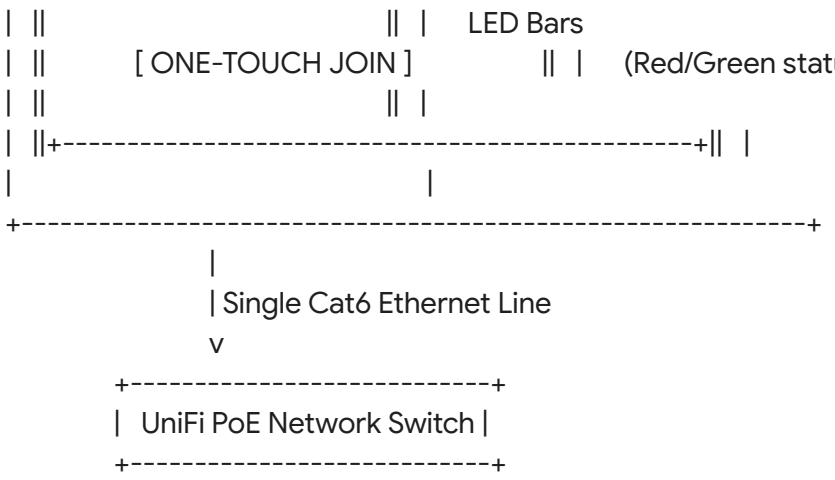
6. Hardware Selection (Path A)

Based on the 24/7 continuous uptime and cabling requirements, the hardware profile uses **Path A: Dedicated PoE Commercial Displays**.

Target Hardware: Mimo Adapt-IQV 10.1" (with Side LEDs)

- **Approximate Retail Price:** $550 - $585 USD
- **Key Strengths:**
  - **Battery-less Construction:** Direct execution off hardwired power prevents safety issues related to prolonged lithium-battery charging.
  - **Unified Cabling (PoE):** Leverages a single RJ45 Cat6 connection back to your **UniFi PoE Network Switch** for both local power delivery and low-latency network traffic.
  - **Visual Side Bars:** Programmatically control built-in side-LED arrays to turn Red/Green using basic Flutter platform-channel calls to Mimo's SDK wrapper.





7. Provisioning & Deployment Playbook

To set up a new hardware node, execute these steps in order:

1. **Build Kiosk Release:**
   Compile the Flutter app using flutter build apk --release from the packages/roombooker_kiosk/ directory.
2. **Device Prep:**
   Install Google Meet, Microsoft Teams, and Zoom native Android applications from the Play Store on the Mimo tablet.
3. **Bypass Zoom Preview Dialog:**
   Open the native **Zoom** application on the tablet manually. Navigate to **Settings > Meetings** and disable **"Show Video Preview Dialog"**. This forces Zoom deep links to auto-join with zero further development work.
4. **Deploy Kiosk Application:**
   Sideload your roombooker_kiosk release APK using ADB, or configure it as a private application in **Managed Google Play** / your MDM launcher.
5. **Grant System Accessibility Permissions:**
   Open the tablet's system settings. Navigate to **System > Accessibility**. Find your registered **MeetAutomatorService** and toggle it to **Enabled**. This allows the app to perform button clicks inside Meet and Teams dynamically.
6. **Activate App Pinning:**
   Launch the RoomBooker Kiosk app, trigger LockTaskMode via your programmatic button or native OS menu pinning to lock the layout screen.